AXI4-AVIP

USER GUIDE

Connecting RTL to the AXI4 AVIP for Verification

Master RTL and Slave RTL bring-up using active and passive agents

1. Overview

An Accelerated Verification IP (AVIP) is a reusable, protocol-aware testbench that knows how to generate AXI4 traffic, respond to it, observe it and check it. To verify a piece of RTL, the RTL is dropped into the AVIP as the Device Under Test (DUT) and the AVIP is configured around it. Two cases occur in practice: the RTL under test is a master, or the RTL under test is a slave. This note describes how the RTL is wired into the AVIP for each case, and explains the job of every block in terms of UVM agents.

AXI4 carries every transaction over five independent channels — write address (AW), write data (W), write response (B), read address (AR) and read data (R). Every channel uses a VALID/READY handshake: the source asserts VALID, the destination asserts READY, and a beat transfers only when both are high. Whether a block has to assert VALID or assert READY on a given channel is exactly what decides whether it must be active or passive in the AVIP.

Because the AVIP supplies both ends of the bus, it can verify a single RTL block on its own (standalone testing) — the AVIP plays the missing side and the scoreboard provides the golden reference. The rest of this document shows how that is arranged.

2. Active and Passive Agents

An agent is the unit in the AVIP that owns one AXI interface. Each agent can be configured in one of two modes, and this single switch (is_active) is what changes from one verification setup to the next:

- **Active agent** — contains a sequencer, a driver and a monitor. The sequencer produces transactions, the driver converts them into pin-level activity and drives the bus (asserting VALID, supplying address/data, honouring handshakes). An active agent therefore stimulates the DUT and behaves like a real bus participant.
- **Passive agent** — contains a monitor only; the sequencer and driver are not built. It never drives the bus. Its monitor passively samples the interface, reconstructs each transaction and forwards it to the scoreboard and coverage. A passive agent is purely an observer.

The rule that ties everything together: the agent that plays the **opposite** role to the DUT is made **active** so that it drives the DUT, and the agent that plays the **same** role as the DUT is made **passive** so that it shadows the DUT and feeds the checker. Keep this rule in mind and the two setups below follow naturally.

3. Master RTL Verification

Here the DUT is the **master**. A master initiates transactions: it drives the AW, W and AR channels and consumes the B and R channels. On its own it cannot run, because every channel needs the other side to handshake. The AVIP therefore has to provide a slave to talk back to it, and an observer to record what it does.

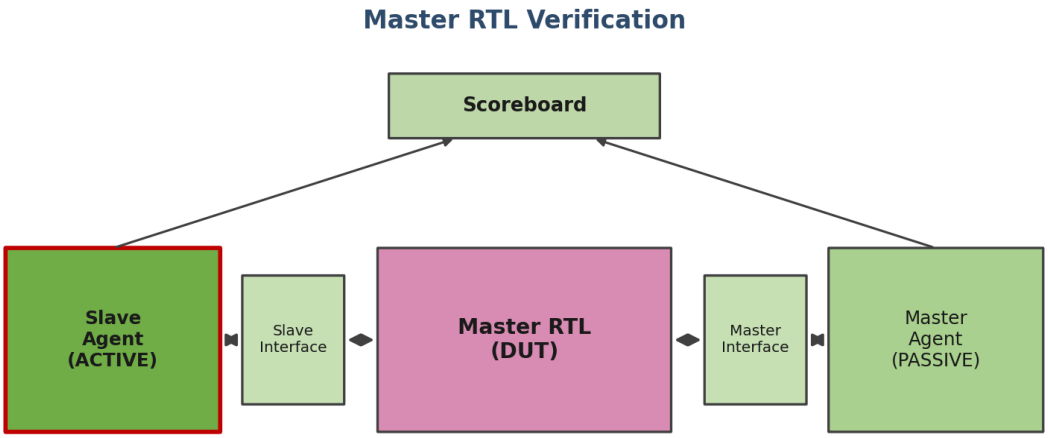

Fig 3.1 Master RTL connected to the AXI4 AVIP

Reading the diagram from the DUT outwards:

- **Slave agent — ACTIVE.** This is the real slave the master DUT communicates with, so it must be active. Its driver accepts the write address (AWVALID/AWREADY), accepts the write data beats (WVALID/WREADY) and returns the write response on the B channel; on the read side it accepts the read address (ARVALID/ARREADY) and drives the read data and response back on the R channel. It also models the slave-side behaviour such as the memory being written to and read from. Without this active agent the master DUT would simply stall, because nothing would assert the ready signals it is waiting on.
- **Master agent — PASSIVE.** It is attached to the same AXI interface as the master DUT but only monitors. Its monitor samples every channel, rebuilds the address, control attributes (id, len, size, burst) and data the master DUT drove, and sends that reconstructed transaction to the scoreboard. This is the verification view of what the master actually generated.
- **Scoreboard.** It receives the transaction observed on the master side and the transaction observed on the slave side and checks they agree — same id, address, length, size, burst type, response and data. A match confirms the master DUT is protocol-correct and moved the intended data; a mismatch points straight at the failing field.

4. Slave RTL Verification

Here the DUT is the **slave**. A slave is reactive: it never starts a transaction, it only responds to one. The roles therefore flip with respect to the previous setup. The side that was passive (the master) now has to be active, because the slave DUT cannot do anything until a master drives a request into it.

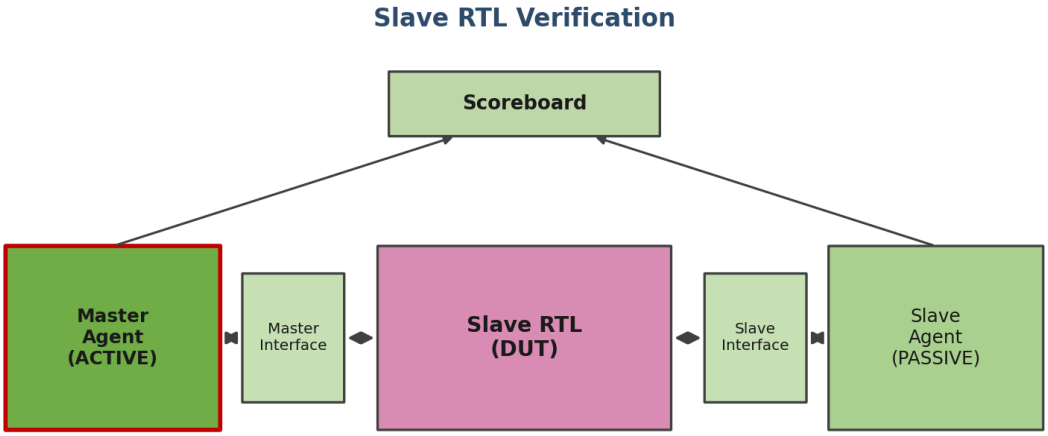

Fig 4.1 Slave RTL connected to the AXI4 AVIP

Reading the diagram from the DUT outwards:

- **Master agent — ACTIVE.** This is the key configuration change. The master agent must be active so that its sequencer generates read and write sequences and its driver drives the AW, W and AR channels into the slave DUT with the chosen addresses, burst lengths, sizes and data. It is the only source of stimulus in this setup. (In the original slide this block appeared as passive; for slave-RTL bring-up it has to be active, otherwise no traffic ever reaches the slave and nothing is exercised.)
- **Slave agent — PASSIVE.** Attached to the same interface as the slave DUT, monitor only. It samples the slave DUT's outputs — the ready signals it asserts, the write response it returns on the B channel and the read data it drives on the R channel — and forwards them to the scoreboard. This captures how the slave DUT actually behaved.
- **Scoreboard.** It compares the request the active master sent against the response the passive slave monitor captured from the DUT, checking id, address, length, size, burst and the read/response data so that what the slave returned is exactly what the protocol and the written contents require.

5. Building and Connecting the Interface in hdl_top

In an AVIP the timed, pin-level parts of the testbench live in **hdl_top**, while the untimed class-based environment lives in **hvl_top**. hdl_top is where the clock and reset are generated, where the AXI4 interface is created, where the RTL under test is wired up, and where the BFMs sit. This

section follows those pieces in order, using a slave RTL as the worked example; a master RTL is wired the same way with its own port names.

5.1 Creating the interface

hdl_top first generates the clock and the active-low reset, then instantiates the AXI4 interface and hands it that clock and reset. The interface bundles the signals of all five channels (awid, awaddr … rready) into one named object, so every other block refers to intf.<signal> instead of to loose wires.

```
module hdl_top;
  bit aclk;
  bit aresetn;

  // generate clock and reset here
  always #5 aclk = ~aclk;

  // 1. create the AXI4 interface and give it clock + reset
  axi4_if intf (.aclk(aclk),
                .aresetn(aresetn));
```

5.2 Mapping the interface to the RTL (master or slave)

The DUT is instantiated right next to the interface and its ports are tied to the interface signals, so the DUT and the AVIP share the exact same wires. For a slave RTL the slave's s_axi_* ports connect to intf.*; for a master RTL the master's m_axi_* ports connect to those same intf.* signals. Whichever role the DUT plays, the interface is the single point where everything meets.

```
  // 2. connect the DUT (slave RTL) to the interface
  axi_ram slave (
    .s_axi_aclk    (aclk),
    .s_axi_aresetn (aresetn),
    // Write Address Channel
    .s_axi_awid    (intf.awid),
    .s_axi_awaddr  (intf.awaddr),
    .s_axi_awlen   (intf.awlen),
    //  … remaining AW / W / B / AR signals …
    // Read Data Channel
    .s_axi_rdata   (intf.rdata),
    .s_axi_rresp   (intf.rresp),
    .s_axi_rlast   (intf.rlast),
    .s_axi_rvalid  (intf.rvalid),
    .s_axi_rready  (intf.rready)
  );
```

For a master DUT the only change is the instance and its port names — e.g. .m_axi_awid(intf.awid), .m_axi_awaddr(intf.awaddr) … — driving the same interface signals.

5.3 Passing the interface to the master BFMs

The BFMs are the driver and monitor that act at pin level on the interface: the driver BFM asserts the channel signals and runs the VALID/READY handshakes, while the monitor BFM only

samples. For the master, these are not placed loosely in hdl_top — they are instantiated and connected inside the **master agent BFM**, which hdl_top brings in on the shared interface.

```
  // in hdl_top: bring in the master agent BFM on the interface
  axi4_master_agent_bfm master_agent_bfm (intf);
```

Inside the master agent BFM the master driver and monitor BFMs sit on the interface, and it is here that the setting of the master driver and monitor BFM handles is done — they are published into the UVM configuration database (uvm_config_db) so the HVL side can locate them by name.

```
module axi4_master_agent_bfm (axi4_if intf);

  // master driver and monitor BFMs on the shared interface
  axi4_master_driver_bfm  master_drv_bfm (intf);
  axi4_master_monitor_bfm master_mon_bfm (intf);

  // set the master driver / monitor BFM handles for the HVL side
  initial begin
    uvm_config_db#(virtual axi4_master_driver_bfm)::set(null,
      "*master_agent*", "axi4_master_drv_bfm", master_drv_bfm);
    uvm_config_db#(virtual axi4_master_monitor_bfm)::set(null,
      "*master_agent*", "axi4_master_mon_bfm", master_mon_bfm);
  end

endmodule
```

5.4 Agent config, and the HVL proxies that call the BFM tasks

On the HVL side each agent reads its configuration. The is_active field decides what the agent builds: an **active** agent builds a driver proxy and a monitor proxy, a **passive** agent builds only the monitor proxy (the same active/passive choice from Section 2). In its build_phase each proxy performs a uvm_config_db get to capture the handle to its BFM that the agent BFM published (for the master, the master agent BFM shown above).

From then on the proxy talks to the RTL purely by calling tasks and functions inside the BFM through that handle. The driver proxy hands a randomized transaction down by calling the BFM's drive task; the monitor proxy calls the BFM's sample task to pull pin activity back up as a transaction for the scoreboard and coverage. This is the bridge between the timed HDL side and the untimed HVL side.

```
class axi4_master_driver_proxy extends uvm_driver #(axi4_master_tx);
  // handle to the master driver BFM set by the master agent BFM
  virtual axi4_master_driver_bfm drv_bfm_h;

  function void build_phase(uvm_phase phase);
    if (!uvm_config_db#(virtual axi4_master_driver_bfm)::get(
          this, "", "axi4_master_drv_bfm", drv_bfm_h))
      `uvm_fatal(get_type_name(), "driver BFM handle not found")
  endfunction
```

```
  task run_phase(uvm_phase phase);
    forever begin
      seq_item_port.get_next_item(req);
      drv_bfm_h.drive_write_address(req);  // call into the BFM
      seq_item_port.item_done();
    end
  endtask
endclass
```

Which BFM actually wiggles the pins follows the active/passive rule: for a slave DUT the master agent is active so its driver BFM drives the interface and the slave agent is passive so only its monitor BFM samples; for a master DUT it is the other way round. The interface, the BFMs and the config_db handoff stay exactly the same — only the agent modes change.

6. Scoreboard and Expected-Behaviour Checking

The scoreboard is the self-checking part of the environment and it generally works in two complementary ways.

6.1 End-to-end comparison

As described above, the scoreboard lines up the transaction seen by the master-side monitor against the transaction seen by the slave-side monitor and compares the protocol fields — id, address, length, size, burst type, response and data. This catches protocol and data errors on whichever side is the DUT.

6.2 Local memory model (golden reference)

For data-integrity checking, and especially for **standalone testing** where only one RTL block sits in the testbench, the scoreboard keeps its own **local memory model** — a reference store (typically an associative array indexed by address). This local memory is what produces the **expected** behaviour, because there is no second live component to compare against; the memory itself is the golden model.

- **On a write transaction**, the scoreboard takes the address and the write data beats (honouring the byte-strobe wstrb) and stores them into its local memory at that address. Nothing is compared yet — the write simply updates the reference image.
- **On a read transaction**, the scoreboard fetches the data previously stored at that address from its local memory and compares it against the read data the DUT actually returned on the R channel. If the DUT read back what was written, the data path is correct; if not, the scoreboard flags the mismatch.

This write-then-read self-checking is what lets a single master RTL or a single slave RTL be verified in isolation: the local memory remembers what every write deposited, so any later read has a known expected value even though no external reference model is present. The same memory model also lets the scoreboard predict read data when the active agent is generating the stimulus, keeping the check valid regardless of which side is the DUT.

7. Summary

The same AVIP is reused for both setups; only the active/passive configuration of the two agents changes with the role of the DUT, and the scoreboard's local memory supplies the expected data when a block is tested on its own.

DUT (RTL)	Agent that drives it (ACTIVE)	Agent that observes it (PASSIVE)
Master RTL	Slave agent — responds to the master's requests	Master agent — monitors the master DUT
Slave RTL	Master agent — generates and drives stimulus	Slave agent — monitors the slave DUT

Note: the active agent always sits on the side opposite the DUT and supplies stimulus, while the passive agent sits on the DUT side and only observes. The scoreboard's local memory provides the golden expected values for write/read checking, which is essential when a single RTL block is verified standalone. Whichever role the RTL plays, swap the two agent modes accordingly and keep the memory-based checking in place.